Workflow and architecture patterns for working together

Jacob Archambault

May 15, 2026

Outline

Outline

Outline

Outline

1. Current challenges and their canonical solutions
2. DevOps 201
    - Communication
    - Throughput
        - DevOps Research and Assessment (DORA)
3. DevOps anti-patterns and applications
    - Anti-pattern 1: local optimization
    - Anti-pattern 2: Gandalf vs. the Balrog
4. Conclusion

Challenges

- coordinating multiple teams working in the same repository

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
    - feedback delays

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
    - feedback delays
    - merge conflict regressions
    - sunk costs fallacy risks

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

Challenges

- coordinating multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams -> Inverse Conway Maneuver

Canonical solutions to these challenges

- multiple teams -> Inverse Conway Maneuver
- collaboration -> Business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams -> Inverse Conway Maneuver
- collaboration -> Business stream-aligned teams over activity oriented teams
- late integration -> adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams -> Inverse Conway Maneuver
- collaboration -> Business stream-aligned teams over activity oriented teams
- late integration -> adopting a true continuous integration workflow
- database coupling -> Bezos API mandate/“Microservices”

Canonical solutions to these challenges

- multiple teams -> Inverse Conway Maneuver
- collaboration -> Business stream-aligned teams over activity oriented teams
- late integration -> adopting a true continuous integration workflow
- database coupling -> Bezos API mandate/“Microservices”

Outline

1967: Conway's law

'Organizations which design systems [...] are constrained to produce designs which are copies of the communication structures of these organizations.' - Melvin Conway, 'How do Committees Invent?' Datamation, 1967

Conway's law: examples

- 'If you have four groups working on a compiler, you'll get a 4-pass compiler' - The New Hacker's Dictionary, 1996

Conway's law: examples

- 'If you have four groups working on a compiler, you'll get a 4-pass compiler' - The New Hacker's Dictionary, 1996
- front-end [devs] business layer [backend devs] monolithic database [DBA team]

Conway's law: examples

- 'If you have four groups working on a compiler, you'll get a 4-pass compiler' - The New Hacker's Dictionary, 1996
- front-end [devs] business layer [backend devs] monolithic database [DBA team]
- a web api controller [manager] delegates most business logic to business classes [developers] which are part of the same in-memory process [team], while serving as a single entry-point for wider cross-network [cross-team] communication.

Conway's law: more examples

- separate .csproj projects for tests written by QA

Conway's law: more examples

- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations

Conway's law: more examples

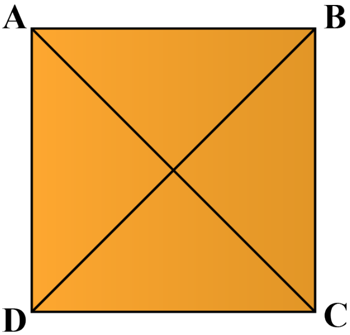
- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations
- all business documentation in the same repository, separate from the repositories that the documentation is for

Conway's law: more examples

- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations
- all business documentation in the same repository, separate from the repositories that the documentation is for

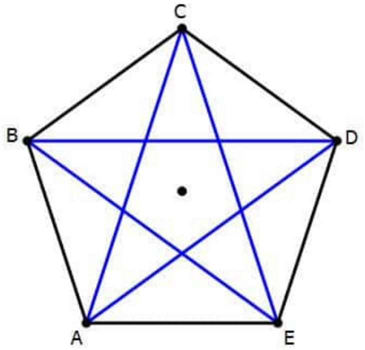
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals$=$ $n(n-1)/2$



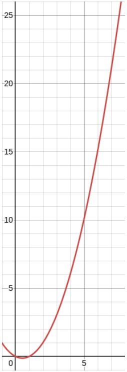
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals$=$ $n(n-1)/2$



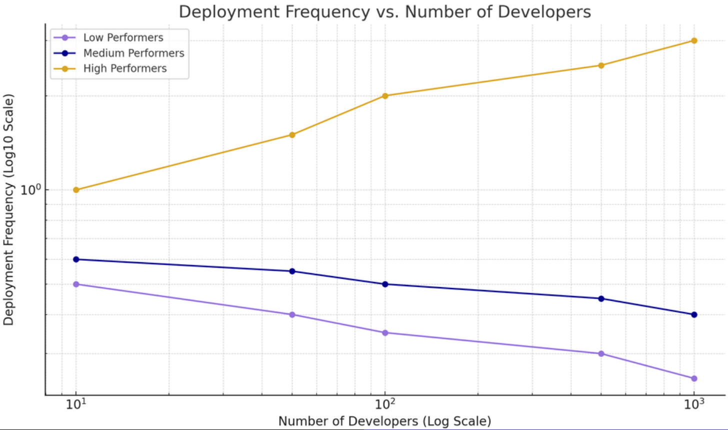
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals= $n(n-1)/2$



The Mythical Man Month, Fred Brooks (cont.)

- corollary: adding more people to a project can lead not only to diminishing returns on delivery speed, but to objectively less work being completed



Outline

1. Current challenges and their canonical solutions
2. DevOps 201
   - Communication
   - Throughput
     - DevOps Research and Assessment (DORA)
3. DevOps anti-patterns and applications
   - Anti-pattern 1: local optimization
   - Anti-pattern 2: Gandalf vs. the Balrog
4. Conclusion

Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput

Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:

Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
    - operating costs

Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
  - operating costs
  - inventory

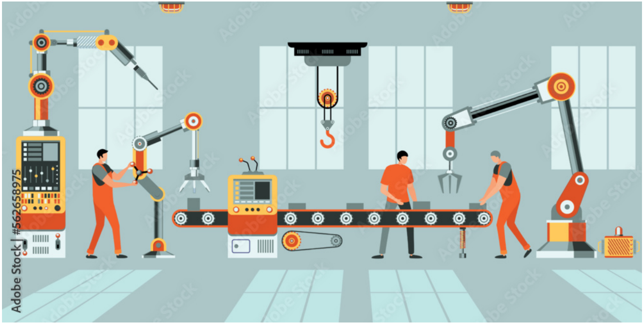
Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
    - operating costs
    - inventory
    - scrap

Just-in-time manufacturing and Goldratt's theory of constraints

- Increase: throughput
- Decrease:
  - operating costs
  - inventory
  - scrap
- Remove bottlenecks

Goldratt's theory of constraints (cont.)



2013: The State of DevOps Report

1. a series of surveys of over 36,000 professionals

2013: The State of DevOps Report

1. a series of surveys of over 36,000 professionals
2. Led by a PhD statistician, Nicole Forsgren, from 2013-2019

2013: The State of DevOps Report

1. a series of surveys of over 36,000 professionals
2. Led by a PhD statistician, Nicole Forsgren, from 2013-2019
3. Uses *cluster analysis* to discover groupings - has no prior understanding of what counts as good or bad.

The State of DevOps Report: throughput metrics

1. lead time for changes

The State of DevOps Report: throughput metrics

1. lead time for changes
2. deployment frequency

The State of DevOps Report: stability metrics

1. change failure rate

The State of DevOps Report: stability metrics

1. change failure rate
2. mean time to restore

The State of DevOps Report: results

'Astonishingly, these results demonstrate that there is no trade-off between improving performance and achieving higher levels of stability and quality. Rather, high performers do better at all of these measures. This is precisely what the Agile and Lean movements predict, but much dogma in our industry still rests on the false assumption that moving faster means trading off against other performance goals, rather than enabling and reinforcing them.' - Nicole Forsgren, Jez Humble, and Gene Kim, Accelerate (2017), p. 14

Outline

Anti-pattern 1: local optimization

- example 1: separate (DevOps, QA, operations, support) team

Anti-pattern 1: local optimization

- example 1: separate (DevOps, QA, operations, support) team
- leads to bottlenecks

Anti-pattern 1: local optimization

- example 1: separate (DevOps, QA, operations, support) team
- leads to bottlenecks
- delays communication

Anti-pattern 1: local optimization

- example 1: separate (DevOps, QA, operations, support) team
- leads to bottlenecks
- delays communication
- punishes untracked work (e.g. thorough code reviews/dev testing, helping blocked colleagues)

Anti-pattern 1 solution

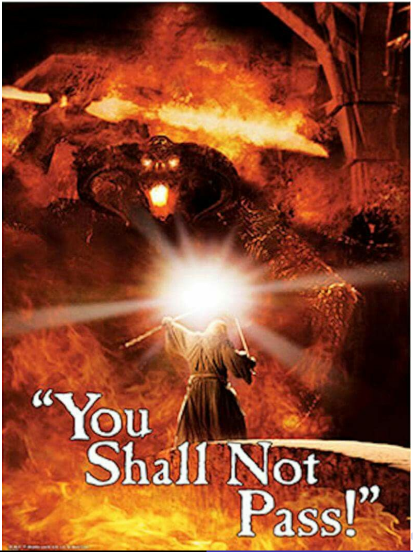
- cross-functional teams with ‘t-shaped’ employees

Anti-pattern 1 solution

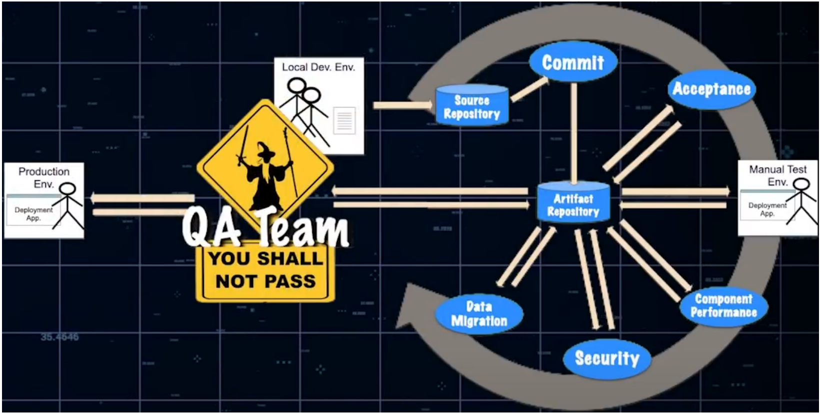
- cross-functional teams with ‘t-shaped’ employees
- guild system

Outline

Anti-pattern 2: Gandalf vs. the Balrog



Anti-pattern 2: Gandalf vs. the Balrog



Anti-pattern 2: Gandalf vs. the Balrog

- change approval boards

Anti-pattern 2: Gandalf vs. the Balrog

- change approval boards
- pre-merge code reviews

Anti-pattern 2: Gandalf vs. the Balrog

- change approval boards
- pre-merge code reviews
- ‘bridge-to-nowhere’ pipelines

Anti-pattern 2: Gandalf vs. the Balrog

- change approval boards
- pre-merge code reviews
- ‘bridge-to-nowhere’ pipelines
- these all increase lead time

Anti-pattern 2 solutions: aggressively parallelize work

- pair programming

Anti-pattern 2 solutions: aggressively parallelize work

- pair programming
- parallelized pipeline builds for ‘wide’ pipelines

Anti-pattern 2 solutions: aggressively parallelize work

- pair programming
- parallelized pipeline builds for ‘wide’ pipelines
- one build artifact for all pipeline stages

Anti-pattern 2 solutions: aggressively parallelize work

- pair programming
- parallelized pipeline builds for ‘wide’ pipelines
- one build artifact for all pipeline stages
- share responsibility

Conclusion

1. Reduce goal misalignment

Conclusion

1. Reduce goal misalignment
2. stop people from waiting on each other

Conclusion

1. Reduce goal misalignment
2. stop people from waiting on each other
3. work in small batches

Conclusion

1. Reduce goal misalignment
2. stop people from waiting on each other
3. work in small batches
4. That's mostly it

Conclusion

'DevOps is whatever you do to bridge friction created by silos, and all the rest is engineering' - Patrick Debois, Puppet State of DevOps report 2021

Workflow and architecture patterns for working together

Jacob Archambault

May 15, 2026